



CEN429 Term Project

Project Guide — 2026-2027 Fall

Asst. Prof. Dr. Uğur CORUH

Goal of the project

In this project you design and build a chosen application **as if it were going through a certification process**:

- a **C++ console application**
- a **dynamic library (DLL/.so)** that provides its security

Goal: combine every security technique you learn during the term into **one product** and document all of it in a **security guide** (S0–S17).

At a glance

- **Team:** at most 3 students (working alone is fine); each topic goes to one team only; no team changes after week 3 (04.10.2026)
- **Language/tools:** C++ (DLL/.so), CMake/CTest, SQLite, SoftHSM/PKCS#11, OpenSSL
- **Two deliveries:** Midterm (RAP1) — first half of the product; Final (RAP2) — full product + test results
- **Heart of the delivery:** alongside the code, a **security guide** written like a certification document (S0–S17)
- All data is **synthetic**; no real secret, key, or personal data may enter the repository
- A detailed scoring key and evidence templates may be shared on Microsoft Teams; **the criteria and points on this page are binding**

Checkpoints

Midterm checkpoint	Week 7 (30.10.2026) · RAP1 — first half of the product
Final checkpoint	Week 15 (25.12.2026) · RAP2 — full product + test results

1. Calendar

Week	Date	Event
End of week 3	04.10.2026	Last day to choose your topic and team
Week 4	09.10.2026	Project plan approved by the instructor
Week 7	30.10.2026	Midterm demo + interim report (RAP1)
Week 8	31.10–08.11.2026	Quiz-1 (weeks 1–6)
Week 15	25.12.2026	Final demo + final report (RAP2)
Week 16	04–17.01.2027	Quiz-2 (weeks 9–14)

The full weekly content plan is in the syllabus. Any date change is announced in the course classroom.

2. Assessment structure — weights

- Midterm = $0.6 \cdot \text{RAP1} + 0.4 \cdot \text{Quiz-1}$
- Final = $0.7 \cdot \text{RAP2} + 0.3 \cdot \text{Quiz-2}$
- Course grade = $0.4 \cdot \text{Midterm} + 0.6 \cdot \text{Final}$

RAP1 (Week 7): first half of the product (S2–S5 + basic protections)

RAP2 (Week 15): full product + S16 results

Learning outcomes (1/2)

LO	Definition
LO.1	Identifies and classifies common software vulnerabilities (buffer overflow, injection, memory leaks)
LO.2	Explains basic encryption methods and secure communication principles (SSL/TLS)
LO.3	Explains code hardening techniques (input validation, secure memory, RASP, obfuscation) and applies them
LO.4	Explains the principles of building secure communication channels using encryption and authentication

Learning outcomes (2/2)

LO	Definition
LO.5	Creates a protection plan using secure software design principles (least privilege, defence in depth)
LO.6	Performs a basic security review and vulnerability assessment
LO.7	Knows secure programming standards (ETSI, EMV, FIPS) and penetration test planning

Both checkpoints touch every learning outcome; RAP1 measures the basic/early level, RAP2 the advanced level and the full product.

3. Tools and setup (1/2)

Tool	Purpose	Output
C++ compiler (GCC/Clang/MSVC)	Building the console app + DLL/.so	Working binary
CMake + CTest	Build system + unit testing	Test report
SQLite	Data-at-rest store	.db schema
SoftHSM / PKCS#11	Key-wrapping/storage simulation	Key-store configuration

3. Tools and setup (2/2)

Tool	Purpose	Output
OpenSSL	Cryptography, TLS, signing	Library linkage + certificates
Doxygen	Source documentation	PDF/HTML output
Git / GitHub	Version control, collaboration	Commit history, pull request
GitHub Actions	Continuous integration (CI)	Green build/test badge

Template repository

Template: `ucoruh/cpp-cmake-ctest-template`

- Fork it, name it `cen429-project-name-surname-cpp`
- Make it **private**
- Add the instructor and your teammate (if any) as **collaborators**

!!! tip "Use the template fully"

The template provides build, CTest unit testing, documentation generation, test/documentation coverage, and packaging. Deliverables that do not follow the template are not accepted. Produce releases; record the version id and digests.

Project layout

- `lib/` — security library (DLL/.so): cryptography, RASP, code obfuscation, data-protection functions
- `app/` — console application: the user interface that uses the library
- `test/` — CTest unit tests (crypto and protection functions)

For environment setup and background knowledge, see the [Prerequisites](#) page.

4. Topic selection

Project topics are in the **Appendix — Project topic list** at the end of the guide page (**100 topics**).

Each topic box gives:

- A short description of the application
- Its key features
- The **assets to protect**
- The **security requirements that stand out** for that topic

How to choose

1. Browse the list at the end of the guide page and choose one topic
2. Record your choice in the **team and topic spreadsheet** on Microsoft Teams — **each topic can be taken by only one team**, the first team to record it gets it
3. Have it approved by the instructor together with your project plan; **once approved, the topic cannot be changed**

Rules for topic selection

- **All** 15 requirements in Section 5 apply to every project; the "highlighted requirements" in a topic box show where they fit most naturally and earn the most credit for that topic
- If you have an idea not on the list, you may choose it with the instructor's approval
- If you are retaking this course, choose a topic **different from your previous project**

5. Requirements

15 requirements · grouped into midterm and final scope

Meet each item, or defer/scope it out with justification (Week 13, S14)

5.1 Midterm scope (RAP1)

4.1 Development environment security

Weeks 1, 12 · S13

- Software development flow and **change management** (baseline → request → classify → approve → release → verify)
- Version control with **Git**; access logging; signed releases
- Development host/server security (short policy)

4.2 Data-in-use security

Weeks 3, 6 · S7

- Sensitive data in memory is **wiped securely** after use (`memset_s` -like)
- Runtime data protection (shadow copy / integrity — Week 6)

4.3 Data-in-transit security

Weeks 3, 10 · S6, S11

- **TLS 1.3**; certificate chain + **SAN** validation; **pinning** + backup pin
- Encrypted **session key**; **device/version binding**; integrity + authentication; server verification code

4.4 Data-at-rest security

Weeks 3, 10, 11 · S8

- File/DB encryption with **AEAD** (AES-GCM); non-repeating nonce
- For sensitive keys, **whitebox** or **SoftHSM/PKCS#11**; **justify** the choice (Week 11)

4.8 Interface definitions and protection

Weeks 1, 4 · S3, S6

- All interfaces protected with access control + authentication
- Input validated at the **trust boundary**

4.9 Code hardening

Weeks 4, 9, 14 · S9

- Opaque loops/predicates, name/file/string/arithmetic obfuscation, opaque booleans, bogus ops/dead branches
- **Control-flow flattening** + random exit, logging **off** in release
- **Measure** the cost (Week 9/14 demos)

4.10 RASP

Week 6 · S10

- Checksum integrity, caller hash/signature verification
- Root/emulator detection, **hook/anti-debug**
- Tamper detection + response, control-flow counter

4.11 Memory protection

Week 4 · S9

- Compiler/OS protections (stack canary, PIE, RELRO, NX, CFI)
- Sensitive memory cleared after use

5.2 Final scope (RAP2)

4.5 Static asset protection

Weeks 1, 4, 9 · S5, S9

Secret keys, digests, source code, resources: **encryption + access control + obfuscation**

4.6 Dynamic asset protection

Weeks 3, 6 · S5, S8

Device/app fingerprints, session data, dynamic keys **are encrypted**

4.7 Asset management

Weeks 1, 3, 13 · S5

Per asset: name, description, location (table/column), source, size, creation/destruction time, default, protection scheme (C/I/I+)

4.12 Cryptography and certificates

Weeks 3, 10 · S8, S11

- Correct algorithm/mode/padding
- Signature verification (==1 , version coverage)
- SSL/TLS + pinning + mutual authentication

4.13 Certification and penetration test plan

Weeks 12, 13 · S16, S14, S17

- Standards mapping (ETSI/EMVCo/GSMA/PCI/MASVS)
- **Penetration test plan** (scope, rules, methodology, test card) and its **results**
- Attack potential + CVSS

4.14 Binary application protection

Weeks 6, 9, 11, 14 · S9, S10, S15

- **Detection** (checksum, anti-debug, emulator)
- **Defence** (obfuscation, string/resource encryption, call hiding)
- **Deterrence** (response/shutdown policy)

4.15 OWASP and build/deploy pipeline

Weeks 5, 14 · S13, S15

- OWASP MASVS/ASVS principles
- **SBOM** (CycloneDX) + dependency scan
- Build pipeline with obfuscation + signing

5.3 Rules for both checkpoints

- All values must be **synthetic**; no real secret, key, or personal data may be in the repository
- Every requirement item must be **visible** in the code and **documented** in the matching S section;
an undocumented implementation counts as unmet
- If a requirement cannot be met, record the **justification** in S14; it cannot be silently skipped
- The application runs only on your own computer (localhost); it must not harm any other system or data

6. Deliverables (1/2)

#	Deliverable	Format
1	Source code	Git repository (fork), C++ DLL/.so + SQLite + SoftHSM
2	Unit tests + CI record	CTest report + CI log
3	Security guide (S0–S17)	.docx / .pdf — midterm: S0–S5, final: complete
4	Test results (S16)	Guide section — final only

6. Deliverables (2/2)

#	Deliverable	Format
5	Compliance matrix (S17) + deferred requirements (S14)	Guide section — final only
6	SBOM + dependency scan	CycloneDX file — final only
7	Release	Git tag: <code>midterm-v1.0</code> / <code>final-v1.0</code>
8	Demo	Live presentation (~10 min) — midterm: Week 7, final: Week 15

Security guide sections (S0–S17)

S0–S8	S9–S17
S0 Cover, version history	S9 Code hardening (advanced)
S1 Scope, abbreviations, references	S10 RASP + response policy
S2 Product and architecture	S11 Secure communication (TLS)
S3 Interfaces	S12 Reporting/logging
S4 Threat model (STRIDE, tree)	S13 Dev process, SBOM
S5 Asset list + protection	S14 Assumptions and deferrals
S6 Identity/binding	S15 Build, signing, deploy pipeline
S7 Data security + shell matrix	S16 Security testing and results
S8 Crypto, key lifecycle	S17 Requirement compliance matrix

Single ZIP archive structure

```
cen429-midterm-name-surname.zip      # final: cen429-final-name-surname.zip
├── cen429-project-name-surname-cpp/  # clone of the GitHub repo
│   ├── lib/                          # security library (DLL/.so)
│   ├── app/                          # console application
│   ├── test/                         # CTest unit tests
│   ├── docs/                         # Doxygen output
│   ├── security-guide/               # S0-S17 security guide
│   ├── sbom/                         # CycloneDX SBOM (final)
│   ├── test-coverage/                # CTest coverage report
└── README.md
```

Naming

- **Repository:** `cen429-project-name-surname-cpp`
- **Archive:** `cen429-midterm-name-surname.zip` / `cen429-final-name-surname.zip`
- **Security guide file:** `cen429-security-guide-name-surname.docx`

The cover page of the report/guide must include the **GitHub repository link**.

7. Team workflow (1/2)

- **GitHub Flow:** `main` is protected and never committed to directly; each task gets its own feature branch, merged through a **pull request**
- **Commit message convention:** [Conventional Commits](#), e.g.
`feat(crypto): add AES-GCM wrapper`, `fix(rasp): correct debugger detection`

7. Team workflow (2/2)

- **Pull request + review:** every PR is **reviewed** by at least one teammate
- **Issues + Projects board:** Backlog → In Progress → In Review → Done
- **No merge without green CI**
- **Version tags:** midterm `midterm-v1.0`, final `final-v1.0`
- **Definition of Done:** builds and runs · unit tests exist · coverage did not drop · PR approved · CI green · guide updated

8. Rubrics

Each criterion is scored on a 1–5 scale

Points = $(\text{level} \div 5) \times \text{criterion points}$

8.1 Achievement levels

Level	Meaning
5 — Excellent	Everything works, is tested and documented
4 — Good	Minor gaps; complete and tested overall
3 — Adequate	Basic operations work; clear gaps remain
2 — Poor	Compiles, but most operations are wrong or missing
1 — No evidence	Not submitted or not working

8.2 RAP1 rubric (Week 7, 100 points)

#	Criterion	LO	Points
1	Threat model and asset list	LO.1, LO.5	15
2	Data security: in transit, at rest, in use	LO.2, LO.4	20
3	Code hardening and memory protection	LO.3	20
4	RASP and response policy	LO.3	10
5	Development environment and change management	LO.5	10
6	Unit tests and CI	LO.6	10
7	Security guide (S0–S5) and demo	LO.7	15
	Total		100

8.3 RAP2 rubric (Week 15, 100 points)

#	Criterion	LO	Points
1	Cryptography and certificates	LO.2, LO.4	15
2	Binary application protection, obfuscation, diversification	LO.3	20
3	Asset management and dynamic assets	LO.5	10
4	Security testing and observed results	LO.6	20
5	Standards, compliance matrix, deferrals	LO.7	15
6	OWASP, SBOM and build/deploy pipeline	LO.1, LO.5	10
7	Report, presentation and demo	LO.7	10
	Total		100

Every row needs **evidence**: a "met" row without evidence counts as unmet.

9. Acceptance conditions

Submissions are not accepted if...

- there is no GitHub repository, it is not **private**, or team members have no commits
- the proportion of requirements met is below the minimum threshold
- the repository or archive contains **binary files** (compiled `.exe` / `.dll` / `.so`)
- no release has been produced, the application does not **build/run** on Windows or WSL/Linux
- the repository contains a real secret, key, or personal data, or **plagiarism** is detected

10. Questions asked in the demo (1/2)

- **Git/GitHub:** Did you fork the template with the correct name? Is the repository private, is the instructor a collaborator? Do both members have commits?
- **Setup and build:** Build the application and the library on Windows and in WSL/Linux; show the
`lib / app / test` split

10. Questions asked in the demo (2/2)

- **Topic (line by line):** Explain your chosen security requirement line by line in the code
- **Tests and documentation:** Open the unit tests and coverage report; show the matching S section
- **File/data operations:** Add a record, close and reopen the program, show that the data comes back encrypted
- **Programming:** Memory management, pointers, compiler/OS protections, debugger inspection

11. Professional responsibility and academic integrity

- You must be able to **explain** every line you submit
- Cite the source if you reuse someone else's code or text
- All examples and values must be **synthetic**
- **Plagiarism** and unauthorized copying cause failure
- A ghost commit (committing on someone else's behalf) counts as plagiarism

Safe and legal frame

Penetration testing is only done on **your own** project.

Examples and values must be **synthetic**; no real secret, key, or personal data may enter the repo.

Your application must not harm anyone else's system or data.

12. Frequently asked questions (1/2)

Can I form a team by myself?

Yes. The team is at most 3 students; teams are fixed at the end of week 3 (04.10.2026).

Can I change my topic after it is approved?

No. Once approved, the topic cannot be changed.

12. Frequently asked questions (2/2)

What happens if I cannot fully meet a requirement?

You may defer it by recording your justification in S14.

Can a requirement left incomplete at RAP1 be finished at RAP2?

Yes, but the RAP1 grade reflects the state at the time of that demo and is not updated afterwards.

Are late submissions accepted?

No — the rules in the syllabus apply.

Appendix — Project topic list

100 topics · 4 groups

- Payments and commerce (01–25)
- Health, education, public services and identity (26–50)
- Media, licensing, communication and the software supply chain (51–75)
- IoT, industry, transport and the enterprise (76–100)

All data is synthetic; the server side is a simulation running only on your own computer (localhost).

Sample topic — 01

Offline Payment Wallet

A wallet that lets a user make small payments without an internet connection. The balance and single-use payment keys live on the device; transactions reconcile with the server once connectivity returns.

Highlighted requirements: 4.4 Data-at-rest (AES-GCM + SoftHSM), 4.3 Data-in-transit (TLS 1.3 + pinning), 4.10 RASP (keys wiped when a debugger is detected)

Sample topic — 26

Personal Health Record Vault

An application where a user keeps synthetic diagnosis, medication and allergy records in a local vault.

The attacker is whoever gains hold of the device or finds a lost one.

Highlighted requirements: 4.4 Data-at-rest (SQLite + AES-GCM + SoftHSM), 4.2 Data-in-use (plaintext kept in memory only briefly), 4.11 Memory protection (a viewed record is wiped once closed)

Where is the full list?

The remaining **98 topics**, each in the same format (short description, key features, assets to protect, highlighted requirements), are in the **Appendix — Project topic list** at the end of the **Project Guide** page.

ucoruh.github.io/cen429-secure-programming/en/project-guide/

Questions

Project guide: ucoruh.github.io/cen429-secure-programming/en/project-guide/

Course site: ucoruh.github.io/cen429-secure-programming